SAT-based Scheduling Algorithm for High-level Synthesis Considering Resource Sharing

Hao Jiang, Fan Yang*, Changhao Yan, Xuan Zeng*
State Key Lab of ASIC and System, Fudan University, Shanghai 201203, P. R. China

2022 IEEE International Symposium on Circuits and Systems (ISCAS) | 978-1-6654-8485-5/22/$31.00 ©2022 IEEE | DOI: 10.1109/ISCAS48785.2022.9937754

***Abstract*—The resource-sharing constraints can be imposed by limiting the maximum allowable number of components for individual functional units in the scheduling process. However, the sharing of the functional units is not explicitly considered in the scheduling procedure. In this paper, we propose a SAT-based scheduling algorithm for high-level synthesis considering the resource-sharing problem. Several pruning strategies have been proposed to reduce the search space. Meanwhile, a minimum-cost network-flow algorithm is proposed to accelerate the repeatedly called scheduling procedure.**

I. Introduction

High-level synthesis (HLS) has been extensively studied recently, due to its high efficiency and reusability [1]. Available commercial and open source HLS tools include Vivado HLS [2], Bambu [3], and LegUp [4] which was once open-source and then commercialized. The scheduling algorithm is one of the most essential components of HLS.

The traditional resource-constrained scheduling imposes the resource-sharing constraints by limiting the number of specific types of resources. List-scheduling [5] and Forced-directed scheduling [6] are well-known heuristics for resource-constrained scheduling. SDC-based scheduling [7] converts the scheduling problem with resource constraints to a system of integer difference constraints (SDC), which can be effectively solved in polynomial time. Nonetheless, it still uses heuristics to deal with resource constraints, leading to suboptimal solutions. A branch-and-bound-based scheduling named BULB [8] utilizes both lower-bound and upper-bound information to prune search space during exploration, thus drastically accelerates the procedure. However, it would also lead to sub-optimal results. A SDS-based scheduling algorithm [9] has been proposed which applies Boolean satisfiability (SAT) to precisely encode resource constraints, and obtains optimal solution by conflict learning.

Traditionally, the scheduling and binding problems are solved individually. The resource-sharing is considered as background information for scheduling through resource constraints. Therefore, the design space is still incomplete in the traditional scheduling approaches. This incompleteness also exists in sharing constrained scheduling where resources can be individually configured such as LPS [10] which formulates the already grouped resource-sharing constraints as a linear programming problem through SDC formulation.

In this paper, we propose a resource-sharing formulation based on SAT [11], aiming to solve the scheduling and binding problems simultaneously as shown in Fig. 1. Several pruning algorithms are proposed to accelerate the design space exploration. Independent scheduling is used to evaluate the quality of both scheduling and resource-sharing results deduced from SAT solution. Generally, the number of independent scheduling calls increases greatly as the number of sharing operation nodes increases. We thus propose a min-cost network flow algorithm to speed up the SDC-based scheduling algorithm. Our algorithm is not a general method, but for certain circuits with high-performance requirements, better results can be obtained than SDS [9]. Our main contributions can be summarized as follows:

*Corresponding authors: {yangfan, xzeng}@fudan.edu.cn.

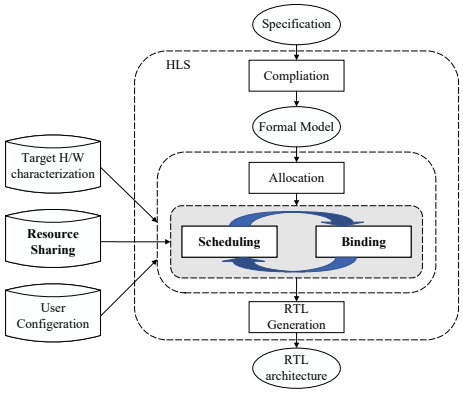

Fig. 1 The resource sharing constraints couple the scheduling and binding problem together in HLS.

(1) We propose a novel resource-sharing formulation based on SAT theory to solve the scheduling problem considering resource-sharing.
(2) We construct the initial clauses of quantity boundary and iteratively transform grouping results into additional clauses, to prune the enormous design space.
(3) We introduce the min-cost network flow algorithm [12] which leverages the original graph structure to accelerate the repeatedly called scheduling process.

The rest of the paper is organized as follows. Section II presents the preliminaries. Section III demonstrates our SAT-based resource-sharing formulation with pruning and the independent scheduling based on the min-cost network flow algorithm. Section IV presents the experimental results, followed by a conclusion in Section V.

II. Preliminaries

A. Scheduling

Given: (1) A control data flow graph (CDFG) $G(V_G, E_G)$ where V_G and E_G represent the set of operation nodes and the interconnections in CDFG, respectively; (2) a set of scheduling constraints, such as dependency constraints, timing constraints, resource constraints, chaining constraints, etc.

Objective: Construct a discrete timing schedule in which each node belonging to V_G has been scheduled to at least one cycle. The constraints must be satisfied simultaneously.

Authorized licensed use limited to: Arizona State University. Downloaded on April 18,2026 at 01:07:29 UTC from IEEE Xplore. Restrictions apply.

B. SDC-Based Approach

Scheduling algorithm based on a system of difference constraints formulation [7] transforms the scheduling problem into an integer linear programming (ILP) problem. In SDC formulation, all operations must follow the constraints expressed in the form of difference as:

$$sv(v_i) - sv(v_j) \leq c_{ij}, \tag{1}$$

where $\forall v_i, v_j \in V_G$ are operation nodes in CDFG, and $sv(v_i)$ is the scheduling variable associated with node v_i. c_{ij} is the constant constraint between v_i and v_j.

The SDC scheduling algorithm employs heuristics based on a total-linear-order set generated by ASAP and ALAP scheduling [7] to solve the resource-constrained scheduling problem. Without considering the propagation influence of the sub-optimal linear order from heuristics, the SDC scheduling obtains the sub-optimal solutions.

C. SDS-Based Approach

A joint SDC and SAT formulation which takes the scalability and optimality into account has been proposed in [9]. SDS-based formulation leverages SDC to evaluate the timing constraints and utilizes SAT solver to deal with the resource constraints.

To exactly formulate the resource constraints, SDS-based formulation introduces sharing variables R_{ij} which represents the resource-sharing relationship between operation node v_i and v_j. R_{ij} is $True$ indicates that operation nodes v_i and v_j are bound to the same resource instance. SDS maps resource-sharing variables to SAT clauses as:

$$\begin{aligned} R_{ij} = True \mapsto & (O_{i\to j} \vee O_{j\to i}) \wedge \\ & \neg(O_{i\to j} \wedge O_{j\to i}) \end{aligned} \tag{2}$$

where $O_{i\to j}$ is the binary partial ordering variables that SDS declares to represent whether v_i is earlier than v_j in scheduling, so as to $O_{j\to i}$. Partial ordering given by SAT solver that maps to difference constraints in SDC as follows:

$$O_{i\to j} = True \mapsto (sv(v_i) - sv(v_j) \leq -1) \tag{3}$$

$$O_{i\to j} = False \mapsto \emptyset \tag{4}$$

Scheduling will be performed with the corresponding resource constraints deduced from SAT results. An iterative process named $conflict\ learning$ [9] explores the feasible design space to find the optimal solution. The SDS-based approach considers both scalability and optimality. However, binding variables which determine the resource-sharing grouping conditions are taken as inputs in scheduling, namely resource-sharing grouping is already known when executing scheduling.

III. SAT-Based Scheduling Considering Resource-Sharing

Precisely encoding resource-sharing is feasible under the condition that resource-binding results are known in advance. Our motivation is to explore the resource binding and grouping strategy simultaneously.

A. SAT-Based Formulation

Considering the resource-sharing problem, we propose a scheduling algorithm that determines an optimal grouping strategy for the operators of the same type and maintains the binding information of hardware resources. The resource-sharing formulation can be expressed as Equation (5), where V_{RS} is a set of operation nodes with the same resource type, and F_{RS} represents grouping results with α_r operation node groups. Each group f_j contains m_j operation nodes. Every node needs to be allocated in one and only one group. α_r denotes the number of resource instances of type $RT(r)$. The scheduler tries to allocate n operation nodes into α_r groups in which operation nodes share the same resource instance.

$$V_{RS} = \{\forall v_i \in V \mid RT(v_i) = r, i = 1, \cdots, n\} \tag{5a}$$

$$F_{RS} = \{f_j \mid j = 1, \cdots, \alpha_r\} \tag{5b}$$

$$f_j = \{v_i \in V_{RS} \mid i = 1, \cdots, m_j\} \tag{5c}$$

$$s.t. \quad \sum_{j=1}^{\alpha_r} m_j = |V_{RS}| = n \tag{5d}$$

$$f_i \wedge f_j = \emptyset,\ \forall f_i, f_j \in F_{RS} \tag{5e}$$

In our formulation, we declare resource-sharing variables without binding results as RS_{ij} which denotes whether operation node v_i sharing the same resource instance with operation node v_j. The fundamental SAT clause can be expressed as:

$$\bigvee_{i=1}^{n_{rs}} \left(\bigvee_{j=i+1}^{n_{rs}} RS_{ij} \right) \tag{6}$$

where n_{rs} denotes the total number of resource-sharing variables in SAT and equals to C_n^2 theoretically. Equation (6) indicates that every two nodes can have a potential sharing relationship.

Actually, each group f_j resembles a $connected\ graph$ with m_j nodes. For that graph, at least $m_j - 1$ edges are essential to maintain the connectedness. The conclusion can be derived that for n nodes to be allocated into α_r groups, the minimum number of edges required is $n - \alpha_r$, i.e. for all SAT variables corresponding to resource-sharing relation, at least $n - \alpha_r$ of them should be assigned to $True$ simultaneously. The above constraints are formulated as follows:

$$F = \bigvee_{l=1}^{C_{n_{rs}}^{n_{tr}}} \left(\bigwedge_{k}^{n_{tr}} x_{lk} \right) \tag{7}$$

where $n_{tr} = n - \alpha_r$ represents the least number of variables that should be assigned to $True$. Equation (7) should be transformed into conjunctive normal form (CNF) which is generally described in $or-and\ expression$. According to Boolean algebras, we can prove that logic expression in Equation (7) can be transformed by the duality principle into expression as follows:

$$F = (F')' = \bigwedge_{l=1}^{C_{n_{rs}}^{n_{var}}} \left(\bigvee_{k}^{n_{var}} x_{lk} \right) \tag{8}$$

where $n_{var} = n_{rs} - (n_{tr} - 1)$, calculated from expression transformation, denotes the constant number of variables in resource-limitation induced boundary clauses.

B. Grouping-Based Pruning

Due to the complexity of SAT solution space, pruning is essential for the exploration of optimal grouping strategy. The entire procedure of pruning is demonstrated in Algorithm 1. The initialization step gathers resource-sharing specifications from scheduling information and calculates all indispensable parameters such as n, n_{RS}, α_r, n_{tr}, n_{var}, etc. The fundamental clause and the $True$-variables boundary clauses are generated to initialize the SAT solver. Then SAT solver gives the value of each sharing variable. After that, the grouping relationship between operating nodes can be derived merely from the sharing variables allocated to $True$.

Authorized licensed use limited to: Arizona State University. Downloaded on April 18,2026 at 01:07:29 UTC from IEEE Xplore. Restrictions apply.

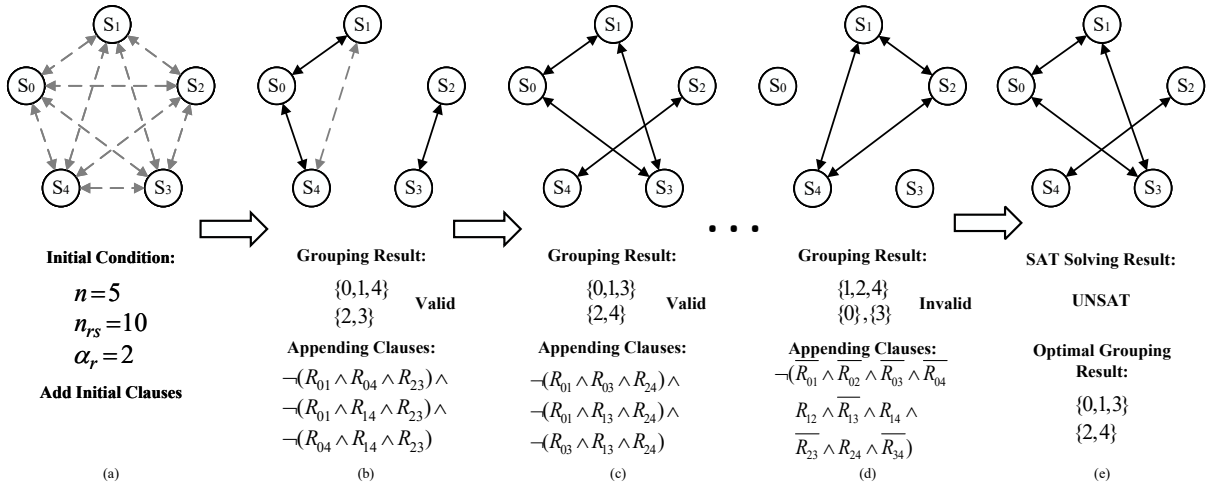

Fig. 2 **Grouping based pruning design space exploration**: (a) initial condition of resource sharing; (b) and (c) valid grouping strategy can be deduced by multiply solutions thus pruning through valid strategy is effective; (d) invalid grouping result leads to a single clause; (e) UNSAT means the optimal solution is included in grouping strategies appeared before.

Algorithm 1 Grouping-based design space exploration

Input: DFG, Constraint set, Resource limitation
Output: Optimal grouping

```
initialization();
add_fundamental_clause();
add_true_num_boundary_clauses();
while result do
    result = SATsolve( solver, &RS_ij );
    valid = grouping( RS_ij, &groups );
    state = encode( groups );
    if Exists( state ) then
        add_clauses( valid, groups );
        continue;
    else
        add_state( state );
    end if
    if valid then
        add_partial_ordering_constraints( groups );
        sch = scheduling();
        cost = update_cost(sch);
        if cost < min_cost then
            update( groups, cost );
        end if
        delete_partial_ordering_constraints();
    end if
    add_clauses( valid, groups );
end while
```

We record the grouping states that are encoded from the SAT solutions. One recorded state namely one grouping result may correspond to many SAT solutions. As shown in Fig. 2, grouping result $\{\{0,1,4\},\{2,3\}\}$ comes from 4 kind of equivalent SAT results: $R_{01}\wedge R_{04}\wedge R_{23}$, $R_{01}\wedge R_{14}\wedge R_{23}$, $R_{04}\wedge R_{14}\wedge R_{23}$ and $R_{01}\wedge R_{04}\wedge R_{14}\wedge R_{23}$.

Thus, if a valid SAT solution's grouping state exists, it means the scheduling based on this grouping result is visited. But, if a new valid grouping state is obtained, we will do the independent scheduling based on the new grouping result. Simultaneously, we will enumerate as many equivalence solutions as possible to prune the search space of SAT solver. On the other hand, if the SAT solution leads to an invalid grouping result, the additional clauses should also be added to the SAT solver to avoid exploring this solution again.

Fig. 2(d) also demonstrates that invalid grouping result is generally deduced from few sharing variables allocated with $True$, like $\{\{1,2,4\},\{0\},\{3\}\}$ which leads to a single additional clause. If we only consider the grouping result of $\{1,2,4\}$ and add several clauses like $\neg(R_{12}\wedge R_{14})$, $\neg(R_{12}\wedge R_{24})$ or $\neg(R_{14}\wedge R_{24})$, then any other grouping results containing $\{1,2,4\}$ (like valid grouping $\{\{1,2,4\},\{0,3\}\}$) will no longer be visited. Therefore, the corresponding grouping result represents a large solution space that cannot be pruned directly.

The independent scheduling with partial-ordering constraints reflects the performance of the grouping results. The objective function which evaluates scheduling is an estimation of the minimum latency. The optimal grouping strategy will be brought to the final scheduling.

C. Scheduling Based on Min-Cost Network Flow

According to Algorithm 1, during the iteration procedure of the entire framework, the independent scheduling process will be tautologically executed many times. Therefore, the acceleration of independent scheduling will significantly improve the overall performance.

To this end, the min-cost network flow (MCNF) algorithm [12] which considers the structure of the data flow graph is introduced to substitute the SDC-based scheduling. Meanwhile, the negative cycle detection algorithm known as the Bellman-Ford algorithm [13] utilized to detect the infeasibility of the independent scheduling can also be performed directly using the same graph data structure with MCNF.

Given a directed acyclic graph (DAG) $G(V,E)$, the min-cost network flow formulation dual-form can be expressed as Equation (9), where f_{ij} represents the $flow$ on edge $e_{ij}\in E$, which is under the limitation of $upper\ boundary\ u_{ij}$ and $lower\ boundary\ l_{ij}$. p_i is a $dual\ variable$ which represents the $price$ attribute adjusted for controlling flow f_{ij}. c_{ij} denotes the $cost$ of per unit flow on edge e_{ij}. Each node $v_i\in V$ corresponds to a variable s_i represents its $supply\ attribute$ which equals to v_i's inflows subtract v_i's outflows. Furthermore, for the whole directed graph, the sum of supply attributes s_i should be zero

Authorized licensed use limited to: Arizona State University. Downloaded on April 18,2026 at 01:07:29 UTC from IEEE Xplore. Restrictions apply.

TABLE I **Synthesis Results Comparison on Benchmarks**

Benchmark	Ops	Inst	Method	F_{max}	ALMs	LUT	Regs	BlockRAMs	RAMs	DSPs
gsm	1108[1]	966[1]	SDC [7]	93.09	3321	4867	5452	10144	7	56
			SDS [9]	94.34	**3233**	4697	5154	-[2]	7	62
			SRS	**98.09**	3236	**4638**	**5150**	10144	7	**54**
mips	433[1]	346[1]	SDC [7]	85.47	1048	1480	928	3072	4	6
			SDS [9]	84.74	**1029**	1468	947	-[2]	4	6
			SRS	**85.79**	1036	**1468**	**926**	3072	4	6
motion	761, 248[1]	625, 199[1]	SDC [7]	109.72	5690	8257	8495	32864	6	0
			SDS [9]	111.11	5729	8339	8958	-[2]	6	0
			SRS	**112.03**	**5680**	**8196**	**8066**	32864	6	0

[1] data of SDC/SRS and multiple data indicates the properties of different functions in the benchmark
[2] block RAMs data of SDS not found

to ensure the flow balance. The min-cost flow problem is to minimize the total cost of the whole graph while guarantees the supply attributes of all nodes [14].

$$\boldsymbol{Max} \quad \sum_{\forall v_i \in V} s_i \cdot p_i - \sum_{\forall e_{ij} \in E} u_{ij} \cdot \alpha_{ij} \tag{9a}$$

$$\boldsymbol{s.t.} \quad \forall e_{ij} \in E: \quad p_i - p_j \leq c_{ij} - \alpha_{ij}(\alpha_{ij} \geq 0) \tag{9b}$$

$$\forall e_{ij} \in E: \quad l_{ij} \leq f_{ij} \leq u_{ij} \tag{9c}$$

$$\forall v_i \in V \ : \sum_{\forall e_{ji} \in E} f_{ji} - \sum_{\forall e_{ik} \in E} f_{ik} = s_i \tag{9d}$$

$$\forall v_i \in V \ : \sum_{v_i} s_i = 0 \tag{9e}$$

Equation (9b) shows that p_i should meet the difference constraint where α_{ij} is the $relaxation\ variable$ [14]. For this reason, p_i can be used to represent the start time of operation node v_i. Similarly, the difference constraints can be represented by cost variables c_{ij} on directed edge e_{ij}.

IV. Experimental Results

The proposed scheduler framework considering resource sharing is implemented by C++. Experiments are conducted on Linux OS with an 8-core Intel Core i7 CPU. We leverage the MiniSat-2.2 [15] solver to figure out the SAT-based resource sharing formulation. An open-source network flow tool named LEMON [16] is utilized to accelerate the repeatedly performed independent scheduling through MCNF-based formulation [12].

A. Scheduling Based on MCNF

The comparison of SDC-based scheduling [7] and MCNF-based scheduling through benchmarks is shown in Fig. 3. Without resource sharing, both algorithms utilize heuristics to handle resource constraints. Furthermore, Fig. 3 demonstrates that MCNF-based scheduling can accomplish up to 33× speedup over SDC-based scheduling. Since the MCNF formulation fully considered the structures of the behavioral description.

B. Scheduling Considering Resource Sharing

We synthesized several benchmarks targeting the Cyclone V FPGA with an objective clock period of 10ns using the proposed scheduling algorithm considering Resource Sharing (SRS for short) and SDC-based scheduler [7], respectively, and compared the implementation results with those of the SDS-based scheduler from [9].

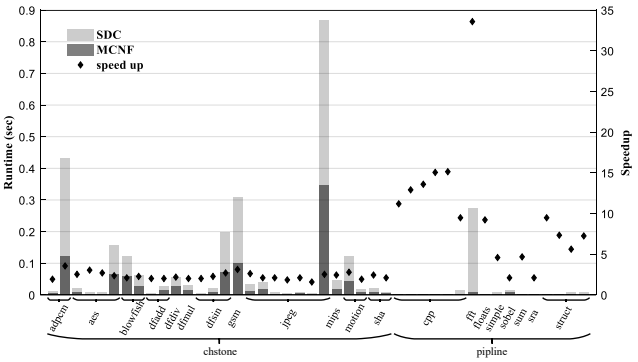

Fig. 3 Runtime comparison between SDC-based and MCNF-based scheduling: bars represent the runtime, and the diamonds represent the speedup ratio.

TABLE I shows the post-synthesis comparison results within the time limitation. Ops and Inst are the number of operation nodes in different functions of benchmark and the number of instructions in benchmark DFG, respectively. F_{max} represents the obtained maximum frequency after synthesis. ALMs, Regs, BlockRAMs, RAMs and DPSs show the utilization of the corresponding resources. Limited by the complexity of the solution space, our algorithm is not a general method, but for certain circuits with high-performance requirements, better results can be obtained than SDS-based scheduling. As shown in TABLE I, the scheduler framework we proposed achieves the better performance in terms of the utilization of logic units and on-chip registers, also the maximum clock frequency.

V. Conclusion

In this paper, we propose a joint scheduling-binding design space exploration framework that considers the resources sharing through SAT-based formulation. We believe it is a good option to further improve the performances of the designs. An MCNF-based algorithm is applied in the proposed framework for repeatedly performed independent scheduling, accomplishing up to 33× speedup over SDC-based scheduling. The synthesized results show that the framework obtains better performance compared with SDS-based scheduling.

Acknowledgements

This research is supported partly by National Key R&D Program of China 2019YFB2205002, partly by National Natural Science Foundation of China (NSFC) 61822402, 62141407, 62090025, 62011530132 and 61929102.

Authorized licensed use limited to: Arizona State University. Downloaded on April 18,2026 at 01:07:29 UTC from IEEE Xplore. Restrictions apply.

REFERENCES

[1] J. Cong, B. Liu, S. Neuendorffer, J. Noguera, K. Vissers, and Z. Zhang, "High-level synthesis for fpgas: From prototyping to deployment," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 4, pp. 473–491, 2011.

[2] H. Ren, "A brief introduction on contemporary high-level synthesis," in *2014 IEEE International Conference on IC Design Technology*, 2014, pp. 1–4.

[3] C. Pilato and F. Ferrandi, "Bambu: A modular framework for the high level synthesis of memory-intensive applications," in *2013 23rd International Conference on Field programmable Logic and Applications*, 2013, pp. 1–4.

[4] A. Canis, J. Choi, M. Aldham, V. Zhang, A. Kammoona, T. Czajkowski, S. D. Brown, and J. H. Anderson, "Legup: An open-source high-level synthesis tool for fpga-based processor/accelerator systems," *ACM Trans. Embed. Comput. Syst.*, vol. 13, no. 2, 2013.

[5] R. Jain, A. Mujumdar, A. Sharma, and H. Wang, "Empirical evaluation of some high-level synthesis scheduling heuristics," in *28th ACM/IEEE Design Automation Conference*, 1991, pp. 686–689.

[6] S. Dai, M. Tan, K. Hao, and Z. Zhang, "Flushing-enabled loop pipelining for high-level synthesis," in *2014 51st ACM/EDAC/IEEE Design Automation Conference (DAC)*, 2014, pp. 1–6.

[7] J. Cong and Z. Zhang, "An efficient and versatile scheduling algorithm based on sdc formulation," in *2006 43rd ACM/IEEE Design Automation Conference*, 2006, pp. 433–438.

[8] M. Chen, S. Huang, G. Pu, and P. Mishra, "Branch-and-bound style resource constrained scheduling using efficient structure-aware pruning," in *2013 IEEE Computer Society Annual Symposium on VLSI (ISVLSI)*, 2013, pp. 224–229.

[9] S. Dai, G. Liu, and Z. Zhang, "A scalable approach to exact resource-constrained scheduling based on a joint sdc and sat formulation," in *Proceedings of the 2018 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, ser. FPGA '18, 2018, p. 137–146.

[10] W. Jiang, Z. Zhang, M. Potkonjak, and J. Cong, "Scheduling with integer time budgeting for low-power optimization," in *2008 Asia and South Pacific Design Automation Conference*, 2008, pp. 22–27.

[11] E. Ábrahám and G. Kremer, "Smt solving for arithmetic theories: Theory and tool support," in *2017 19th International Symposium on Symbolic and Numeric Algorithms for Scientific Computing (SYNASC)*, 2017, pp. 1–8.

[12] P. Kovács, "Minimum-cost flow algorithms: an experimental evaluation," *Optimization Methods and Software*, vol. 30, no. 1, pp. 94–127, 2015.

[13] V. Faychuk, O. Lavriv, M. Klymash, V. Zhebka, and O. Shpur, "Investigation of the bellman-ford algorithm enhanced for remote execution," in *2019 3rd International Conference on Advanced Information and Communications Technologies (AICT)*, 2019, pp. 204–208.

[14] R. Jaillet, "Network flows: Theory, algorithms, and applicationsby ravindra k. ahuja; thomas l. magnanti; james b. orlin," *Transportation Science*, vol. 28, no. 4, pp. 354–356, 1994.

[15] N. Een, A. Mishchenko, and N. Amla, "A single-instance incremental sat formulation of proof- and counterexample-based abstraction," in *Formal Methods in Computer Aided Design*, 2010, pp. 181–188.

[16] B. Dezső, A. Jüttner, and P. Kovács, "Lemon – an open source c++ graph template library," *Electronic Notes in Theoretical Computer Science*, vol. 264, no. 5, pp. 23–45, 2011, proceedings of the Second Workshop on Generative Technologies (WGT) 2010.

Authorized licensed use limited to: Arizona State University. Downloaded on April 18,2026 at 01:07:29 UTC from IEEE Xplore. Restrictions apply.